Dry Run

Input array = [1, 6, 11, 5]
Total Sum (S) = 1+6+11+5 = 23
Target Subset Sum = (S/2) : 23/2 = 11
bool dp[12] = {false}

dp[0] = true

i = 1 current element = 1
j = 1 → sum possible Yes if sum (1-1=0) possible
dp[0] = true → dp[1] = true

0	1	2	3	4	5	6	7	8	9	10	11
T	T	F	F	F	F	F	F	F	F	F	F

i = 2 current element = 6

j = 7 is sum of 7 possible ? Yes if sum of 7-6=1 possible
dp[1] = true → dp[7] = true

j = 6 is sum of 6 possible ? Yes if sum of 6-6=0 is possible. dp[0] = true → dp[6] = true

0	1	2	3	4	5	6	7	8	9	10	11
T	T	F	F	F	F	T	T	F	F	F	T

i = 3 Current Element = 11
j = 11 possible if 11 - 11 = 0 possible
dp[11] → true

0	1	2	3	4	5	6	7	8	9	10	11
T	T	F	F	F	F	T	T	F	F	F	T

i = 4 Current Element = 5

j = 11 dp[11] or dp[11-5] → true
j = 6 dp[6] or dp[6-5] → true
j = 5 dp[5] or dp[5-5] → true

0	1	2	3	4	5	6	7	8	9	10	11
T	T	F	F	F	T	T	T	F	F	F	T

Calculating minimum difference

(i) Now we have processed all elements, we look at final dp array

(ii) We want to find the largest index j (from 11 to 0) where dp[j] is true

(iii) Scanning backwards from 11, dp[11] is true

(iv) A subset with sum of 11 is true Subset 1: (6, 5)

(v) other is (1, 11) sum = 12

(v) Formula = -2 * j + Total Sum

(vi) 23 - 2 * 11 = 1